Recommender Systems - Course Project Report

Cumulative report template - Parts 1, 2, and 3

Team	Group 5
Members	Laura Karjalainen, Tommi Lamberg and Kush Aswani
For Part 3 track: select one	A / B / C
GitHub repository link	https://github.com/lauraKarj/DATA.ML.360-Group-5-project.git

How to use this template

- *This is one cumulative report. Extend and revise the same document at each project stage.*
- *Maximum length: 4 pages at Part 1, 8 pages at Part 2, and 12 pages for the final report, excluding references.*
- *Keep the section headings below. You may add subsections where useful.*
- *All explanations and justifications requested in the project specification belong in this report.*
- *The Results Sheet contains the complete standardized numerical record. In the report, include only the tables and figures needed to support your analysis.*
- *Delete all italic instructional text before the final submission.*

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

In the dataset, there are 6040 users in 'users.dat', 3883 movies in 'movies.dat', and 1000209 ratings in 'ratings.dat' data frame. Density is 0.042647, meaning around 4.3% of all possible interactions have actually happened. That way, 4.3% of the matrix is filled, and 95.7% is empty. Usually, the density of real matrices is only 1%, so our matrix is quite dense, over four times denser than that. (Lecture 1.1) Even though there were 3883 movies, only 3706 of them were found among the ratings, so approximately 95% of listed movies were rated and 5% weren't. For each user there is UserID, Gender, Age, Occupation and ZIP-code. For each movie there is MovieID, Title and Genres and for each rating there is UserID, MovieID, Rating and Timestamp.

Figure 1 shows a distribution of ratings per user. Can be observed that the majority of users have given only a small number of ratings which creates the head of the distribution. However, there is a long and narrow tail for distribution which consists of most of the information. In the tail, there are all of the users which have given most of the ratings, up to 2314 ratings per user. Figure 1 also shows a long-tail distribution of ratings per movie. Can be observed that the majority of movies have received only a small number of ratings which creates the small head also for this distribution. The long tail of the distribution consists of movies with most ratings, up to 3428 ratings per movie. Since the tails of both distributions contain the highest numbers of ratings as well as valuable information, we must protect those. Both distributions are highly skewed due to a large number of users and movies with small rating counts, making them heavily weighted toward the left.

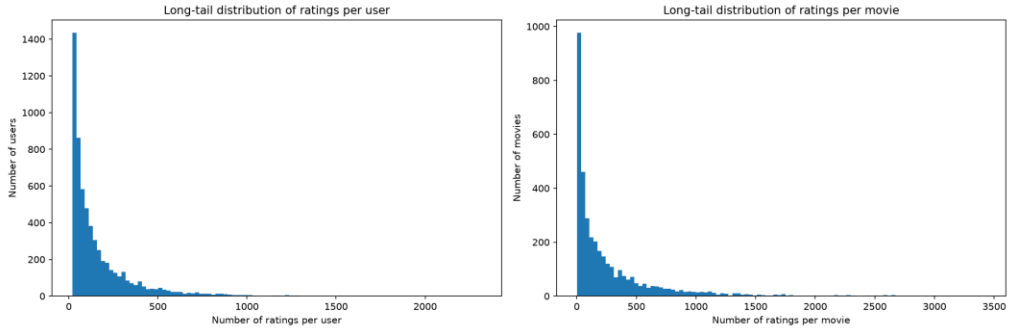


Figure 1 Distribution of ratings per user & movie

In Figure 2, there is a distribution of ratings. Only ratings 4 and 5 are considered positive values. The rating 4 has been given the most: around 350 000 times and rating 5 around 225 000 times.

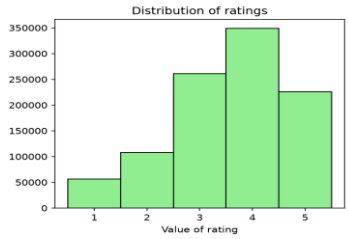


Figure 2 Distribution of ratings

In Figure 3, distribution of rating activity over time is plotted. Can be observed that the majority of the ratings have been given during the year 2000 when ratings given varied greatly, and there are two large peaks visible. For years 2001-2003, number of ratings drops tragically and remains consistently low without peaks. As can be seen, the peaks are not due to a single month during which, in total, more reviews were submitted compared to other months. In November 2000 there is a large peak around the middle of the month which covers maybe only a few or a couple of days. In those days almost 120 000 ratings were given, which is almost half of the ratings given in whole month. This is extraordinary in the whole data.

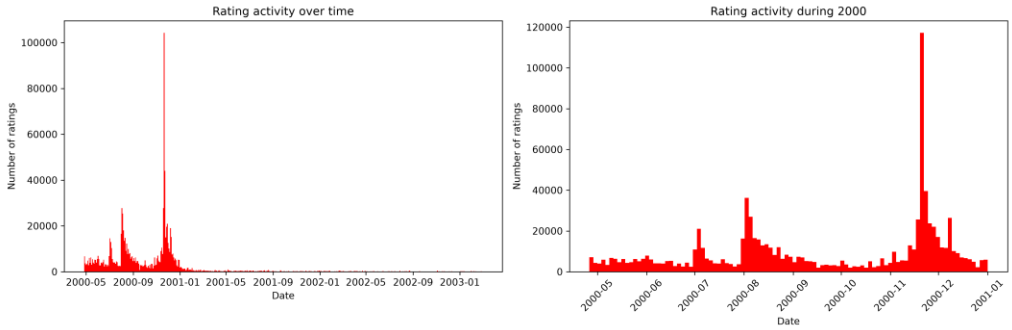


Figure 3 Histogram of rating activity over time and during 2000

1.2 Preprocessing and feedback treatment

All users contained in the dataset have given at least 20 ratings. However, there are movies without ratings. We chose that every movie should have at least 5 ratings and the rest is filtered out. By keeping the movies with at least 5 ratings we don't cut too much of the head and can preserve the tail of the distribution and also users wouldn't lose ratings as much. That way we don't lose the majority of our movies, and we can continue to provide users with recommendations. (Lecture 5.1, slide 5) After the cut, there are a total of 3416 movies left. There weren't any duplicates or null values for IDs. Because filtered out movies, each user doesn't have at

least 20 ratings anymore but instead of at least 18 ratings. In total, there are 999 611 ratings left, so the filtering removed 598 rows.

We treat the ratings as implicit feedback with a relevance threshold of 4: a rating of 4 or 5 is a positive interaction and, in the held-out data, a relevant item, while ratings 1–3 are not used as model input. 4–5 stars express liking while 3 is neutral, and the task is top-N recommendation evaluated with binary-relevance ranking metrics (Lecture 3.1, slides 7–8), so a threshold is needed anyway. Using the same one for the model input keeps training and evaluation consistent. Positive interactions are not rare: 57.5% of all ratings are 4 or 5. The choice was also better on validation: item-kNN with implicit input reached 0.186 NDCG@10 against 0.173 with explicit ratings, and Most Popular counting only ratings ≥ 4 beat counting all ratings (0.170 vs 0.166). The explicit variant gave higher catalogue coverage, which we revisit in Part 2

1.3 Train/validation/test split

From Figure 3, it can be seen that the ratings are unevenly distributed over time. Many ratings have been given in 2000 while in the years 2001-2003 the number of ratings is significantly low. For that reason, we can't just take a train set to be 70% based on the timeline. The temporal split is done by giving 70% of all ratings to training, 20% for validation, and 10% for testing. The training set receives the largest proportion because the model needs sufficient data to learn about users' preferences and movies' patterns. For validation, 20% is used so a trained model can be evaluated and tuned with sufficient amount of data. Even though the model does not directly learn from the validation data, the model can become biased towards it. For that reason, there should be a test data set which remains untouched until the model is validated. Since the data is so large, we decided that 10% is still enough data for testing. Exact timestamps were computed, and results and splits are listed in Table 2. From Table 2 can be seen that the train set covers around 80% of users, so around 20% of users haven't given ratings in the training period. However, the validation data includes almost all users, including users who started rating after 22 November 2000.

Table 1 Timestamps and splits

Timestamp	Ratings given	Users	Set
22.11.2000	69.51 %	80.23 %	Training ~70 % | Until 22.11.2000
23.11.2000	71.64 %	81.99 %	Validation ~20 % | 23.11.2000-31.12.2000
31.12.2000	90.25 %	99.74 %	Validation ~20 % | 23.11.2000-31.12.2000
1.1.2001	90.46 %	99.90 %	Test ~10 % | From 1.1.2001

1.4 Evaluation protocol

Candidate set was all 3416 movies except those the user has already rated. We do not sample negatives, because the brief asks for ranking against the full catalogue and sampled candidates can change which model looks best in the held-out period and at least one rating in history. Because many users joined during the validation period (§1.3), this leaves 608 users on validation, with 1082 users excluded for having no training history, and 1150 users on test.

Precision@10, Recall@10 (hits divided by all of the user's relevant movies) and NDCG@10 with binary gains are averaged over users (Lecture 3.1). Catalogue coverage is the share of the 3416 movies that appear in at least one top-10 list. Movies the user has already rated are never recommended, and ties are broken deterministically. We implemented the metrics ourselves and checked them against scikit-learn's

precision_score, recall_score and ndcg_score on 200 randomly sampled test users (seed 42); the largest difference was $2 \cdot 10^{-16}$.

1.5 Baseline methods and settings

Item-based kNN (Lecture 1.2) scores an unseen movie i for user u as

score(u, i) = Σ_{j ∈ N_k(i) ∩ I_u} sim(i, j) · w_uj,

where N_k(i) is the set of the k movies most similar to i (positive similarities only) and I_u is the user's history. In the implicit variant w_uj = 1; in the explicit variant it is the rating minus the user's mean. We use a plain sum rather than the weighted average used for rating prediction, because the average gives a movie with one weak neighbour the same score as one with many strong neighbours, which hurts top-N ranking (Deshpande & Karypis, 2004). Similarities are computed on the training matrix with missing ratings as 0; the model is deterministic.

We searched 54 configurations: implicit feedback (rating ≥ 4 → 1) with cosine and Jaccard; explicit ratings with cosine, adjusted cosine (user mean subtracted), Pearson (movie mean subtracted) and Jaccard; each with k ∈ {5, 10, 20, 50, 100, 200, 400, 800, all}. Each was trained on train and scored on validation. The configuration with the best validation NDCG@10 (Lecture 3.1) was retrained on train + validation and evaluated once on test. Differences are judged with 95% bootstrap confidence intervals over users (2000 resamples, seed 42).

Implicit similarities beat explicit ones, because the target (a future rating ≥ 4) is about which films a user chooses and likes, which co-occurring positive ratings capture directly. Among explicit measures only adjusted cosine was competitive (0.173), since subtracting the user's mean removes the gap between generous and strict raters (Sarwar et al., 2001); plain cosine treats even a 1-star rating as similarity. Pearson was worst (≤ 0.140): subtracting the movie's mean removes its overall appeal, so it recommends the least popular movies of any configuration. Among implicit measures, Jaccard was better for k ≤ 200 and cosine for k ≥ 400, as Jaccard penalizes niche–popular pairs more.

Implicit cosine with k = 800 had the highest validation NDCG@10 (0.186; Picture 5). Accuracy rises with k up to about 400 and then levels off. Every k ≤ 200 is significantly worse than k = 800, k = 400 is borderline (−0.005, CI −0.010 to 0.000), and using all neighbours is indistinguishable (−0.001, CI −0.005 to 0.003). The data therefore support "a large neighbourhood" rather than exactly 800. We keep k = 800 as the validation optimum under our pre-declared criterion, and because it covers more of the catalogue than no pruning (9.0% vs 6.8%).

1.6 Results and analysis

Two effects shape Figure 4. With a small k most movies get no score (383 scored unseen movies per user at k = 5, about 3100 with all neighbours), so the top 10 comes from a small, noisy pool. A larger k pulls the lists toward popular movies. For implicit cosine, from k = 5 to k = 800 the recommended movies' average number of training ratings rises from 1082 to 1495, and coverage falls from 20.5% to 9.0%, moving toward Most Popular (1876 ratings, 3.7%). Popular movies are similar to many others and collect score from almost every profile, so part of the gain is popularity mixed into the personal score. Precision rises (0.128 → 0.170) while Recall@10 barely moves (0.055–0.061). Test scores exceed validation scores (0.246 vs 0.186) because test users have longer histories and are a different group (§1.3), not because the model improved.

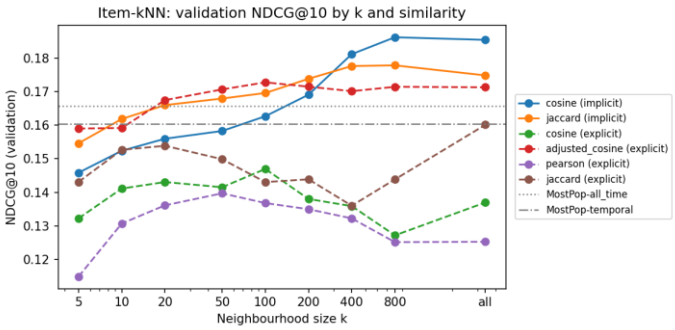


Figure 4 Validation NDCG@10 of item-kNN by neighbourhood size k and similarity; dotted lines are the Most Popular variants

Two Most Popular baselines rank movies by their number of training ratings: all-time uses every rating, temporal only the most recent 20% by timestamp. Both ignore the rating value, so much-watched but poorly rated movies can be recommended; counting only positive ratings (≥ 4) would match our feedback treatment. All-time is slightly better on validation (NDCG@10 0.166 vs 0.160), and temporal on test (0.210 vs 0.201), though with lower coverage (3.8% vs 4.3%). On validation, Most Popular beats every explicit cosine, Pearson and Jaccard setup and implicit cosine up to k = 100; only adjusted cosine and implicit Jaccard with k ≥ 50, and implicit cosine with k ≥ 200, clearly beat it.

Against all-time Most Popular, the stronger variant on validation, item-kNN gains 0.045 NDCG@10 on test (95% bootstrap CI 0.036–0.055; Wilcoxon $p < 0.001$) and 0.020 on validation (CI 0.008–0.033). The win is not universal: on test, kNN gives the better list for 46% of users and popularity for 25%, and for 28% neither finds a relevant movie. The gain grows with activity (+0.038 to +0.051 across history-size quartiles) and is largest for users with the least mainstream taste (+0.060 vs +0.037; mainstreamness is the mean rating count of the movies in a user's history). On validation the least active quartile gains only +0.009, and kNN cannot serve the 1082 users without a history. kNN's advantage is therefore concentrated in users with long, non-mainstream histories; these subgroup numbers are descriptive, and Part 2 will test them. Against temporal popularity, the test gap is smaller (0.246 vs 0.210).

2. Part 2 - Models and Honest Comparison

2.1 Models

Describe the additional models and paradigms. Clearly identify the self-implemented algorithm and explain its implementation.

2.2 Hyperparameter tuning and variability

Describe the validation-only tuning procedure, search spaces/budgets, random seeds, and confidence intervals or statistical testing.

2.3 Overall results

Present and discuss the main comparison. Explain important differences and avoid unsupported claims.

2.4 Disaggregated analysis

Report and interpret results broken down by at least two of the required dimensions: items with few vs. many training interactions, low- vs. high-activity users, popular vs. long-tail items, and early vs. late test period. Define any thresholds used and focus on findings that change or qualify the overall conclusions.

2.5 Sparsity ablation

Report and discuss the 100%, 50%, and 20% training-interaction experiments, including changes in the ranking between models.

3. Part 3 - Beyond Accuracy

3.1 Selected track and motivation

State Track A, B, or C and briefly motivate the selected intervention/extension.

3.2 Method

Describe only your selected track. The Track A/B/C guidance below is provided as instructions; delete the guidance for the two tracks you did not select before submission.

If Track A - Diversity and popularity bias

State the base recommender, selected re-ranking method, genre-vector item representation, trade-off parameter, and exact item-distance measure. Describe how the parameter was swept.

If Track B - Fairness

Define the release-decade provider proxy, report and justify the final decade boundaries and group sizes, state and justify the adopted fairness notion, describe the consumer groups based on MovieLens user attributes, and identify the stage and form of the mitigation.

If Track C - Group recommendations

State the selected synthetic group type and justify the chosen group size (3-5). Describe the three aggregation strategies. Identify which strategy was extended to the sequential scenario and explain how satisfaction is carried forward between recommendation rounds.

3.3 Results and trade-offs

Present the required track-specific results. Track A: report intra-list diversity, catalogue coverage, Gini and calibration error across the parameter sweep, and discuss the accuracy-diversity frontier. Track B: report position-weighted exposure against catalogue share, per-group consumer-side accuracy, and the fairness-performance frontier. Track C: compare group satisfaction, disagreement and minimum member satisfaction across the three aggregation strategies, then analyse how member and group satisfaction develop in the sequential scenario.

3.4 Cost and interpretation

State what is gained, what is lost, and who bears the cost where applicable. Explain what operating choice or strategy you would select and why.

3.5 "One number we cannot explain"

Identify one result you do not fully understand and describe how you would investigate it.

4. Discussion and Conclusions

Summarize the main findings across the project. Discuss limitations, failed approaches where relevant, and what the experiments support.

5. Contribution Statement

For each team member, provide two or three sentences describing their contribution.

Tommi Lamberg (Part 1): Wrote the shared evaluation module. Implemented item-based kNN, ran the 54-configuration search on validation and the final test evaluation, and analyzed the choice of k and similarity and the comparison with Most Popular (Figure 4). Wrote the report sections on evaluation, item-kNN and the kNN-vs-popularity comparison.

Laura Karjalainen (Part 1): Initialized the project structure, wrote README and data preparation module. Loaded data, did characterizing and plotted distributions for section 1. Filtered data, implemented, and justified the split of the data. Wrote sections of data characterization, test/validation/test split and part of preprocessing and feedback treatment.

Kush Aswani (Part 1): Implemented the most popular baseline models (most popular all time and temporal models) . Wrote report section on justification and evaluation of the most popular baseline models.

6. Use of AI Assistants

State which AI tools were used and for what. If none were used, state that explicitly.

Tommi Lamberg (Part 1): Used GrammarlyAI for fixing grammatical errors in text, Claude to help debug an issue with knn (self-similarity was left in originally)

Laura Karjalainen (Part 1): Used ChatGPT to help find and fix bugs in code. Also asked for help with writing instructions to Linux/macOS in README since I have Windows personally.

References

List references used in the report. References do not count toward the page limit.

Deshpande, M., & Karypis, G. (2004). Item-based top- N recommendation algorithms. *ACM Transactions on Information Systems*, *22*(1), 143–177. https://doi.org/10.1145/963770.963776

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. *International World Wide Web Conference (WWW)*, 285–295. https://doi.org/10.1145/371920.372071